



Ju-cheol Park

프론트엔드 개발자, 박주철입니다.

 jukrap628@gmail.com

 <https://github.com/jukrap>

 <https://jukrap.vercel.app>

 <https://www.linkedin.com/in/jukrap>



박주철

jukrap628@gmail.com

About Me

👋 Hi there, I'm **Frontend Engineer**.

화면 너머의 흐름까지 확인하는 프론트엔드 개발자, 박주철입니다.

1. 사용자가 실제로 끝까지 사용할 수 있는 기능을 만들고자 합니다.

배송 운영 웹과 모바일 라벨 출력 기능을 개발하며, 화면에서 버튼이 동작하는 것만으로는 충분하지 않다는 것을 배웠습니다. 예약 접수, 엑셀 업로드, 라벨 출력, 모바일 권한, 장비 선택, 출력 결과까지 이어지는 흐름이 맞아야 실제 업무에서 사용할 수 있었습니다. 이 경험 이후로는 기능을 만들 때 사용자가 어떤 순서로 쓰는지, 어디서 막힐 수 있는지를 먼저 확인하려고 합니다.

2. 상태와 데이터 흐름이 드러나는 코드를 작성하려고 합니다.

React와 TypeScript를 사용해 여러 업무 화면을 개발하면서 조회, 변경, 화면 상태가 섞이면 작은 수 정도 예상하기 어려워진다는 것을 느꼈습니다. TanStack Query로 서버 데이터 갱신 흐름을 정리하고, Zustand로 모달과 테이블 같은 화면 상태를 분리하며 유지보수하기 쉬운 구조를 고민했습니다.

3. 낯선 문제도 실제 조건에서 확인하며 좁혀갑니다.

모바일 WebView와 Android 네이티브 모듈을 연결해 Bluetooth 프린터 출력을 구현할 때는 브라우저 안의 문제만 보서는 해결되지 않았습니다. Android 권한, WebView bridge, 장비 SDK, 라벨 데이터 형식을 실제 기기에서 하나씩 확인하며 원인을 줄였습니다. 자세선생 프로젝트에서도 OpenCV 기반 구현의 성능 문제를 Mediapipe로 전환하며 개선한 경험이 있습니다.

Skills

Frontend

React, Next.js, Vite, TanStack Query, Zustand, Tailwind CSS

Mobile

React Native, Expo, Android Java, Gradle

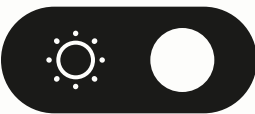
Tooling/Ops

Node.js, Firebase, Google Cloud Platform, Vitest, Jest, Storybook, Sentry, AWS

Career

트리포스(주)
프론트엔드 엔지니어
2026.02 ~ Present

- React 기반 업무 화면, 모바일 WebView, Android 연동처럼 사용자 흐름과 시스템 경계가 맞물리는 영역을 설계하고 구현합니다.
- 신규 구축과 레거시 개선을 함께 다루며 상태, 라우팅, 출력, 검증 기준을 코드와 문서로 남깁니다.



Activity

스터디 운영

2024.05 ~ 2025.06

코딩 테스트 및 개발 지식 스터디

- 총 2개 스터디(코딩 테스트, 개발 지식)를 운영하였습니다.

프로그래머스

데브코스

2024.05 ~ 2024.09

Cloud Application Engineering 과정 - 서브멘토

- 이전 데브코스 기수에서 우수 수료자로 선정되어, 해당 과정 2기 수강생들에게 멘토링을 해주는 서브 멘토로 활동하였습니다.
- 멘토링 분야 : 매주 개발 정보 공유, 수료생 고민 상담, 데일리 스크럼 참여, 개발 문제 해결, 개발 프로젝트 점검 등

스터디 멘토

2021.09 ~ 2022.02

멘토-멘티 코딩 멘토링

- 멘토로서 멘티에게 프로그래밍 및 개발 분야를 지도해주는 스터디에 참여하였습니다.

Awards

DIVE 2024

해커톤

2024.10

부산테크노파크원장상 수상

- 출품 작품 : 동해선장
- [발제사 3위](#)

경상대학교

인공지능 미술전

2022.11

장려상 수상

- 출품 작품 : 석양
- Stable Diffusion 사용

코딩역량강화 교내대회

2021.10 ~ 2021.11

개척상 수상

- 출품 작품 : ESD 핫딜

경남소프트웨어경진대회

2021.08 ~ 2021.10

최우수상 수상

- 출품 작품 : ESD 핫딜
- [“도내 소프트웨어계 이끌 인재들입니다”](#)

Education

프로그래머스

데브코스

2023.12 ~ 2024.05

Cloud Application Engineering 과정 - 수강생

- React와 React Native 개발을 목표로 하는 Cloud Application Engineering 데브코스 과정을 수료하였습니다.
- 훈련 분야 : React & React Native

경상국립대학교

2019.03 ~ 2023.02

컴퓨터과학과

- 대학교(학사) 졸업



🚚 물류 관리-운영 웹

2026.04 ~ 2026.06

조회, 예약, 엑셀, 출력 흐름을 갖춘 물류 업무 웹을 구축했습니다.

물류 조회, 예약 접수, 현황, 주소록, 엑셀 처리, 라벨 출력까지 이어지는 관리-운영 웹을 처음부터 구축했습니다. 기능 수를 늘리는 것보다 PC와 모바일에서 같은 업무 기준으로 동작하는 흐름을 만드는 데 중점을 뒀습니다.

기술 스택

TypeScript · React · Vite · React Router · TanStack Query · Zustand · XLSX · Vitest

프로젝트 종류

Web

프로젝트 기반

신규 구축

2,405.50 kB → 616.59 kB

- [초기 로드 번들]
- 라우트 단위 코드 분리와 엑셀 처리 라이브러리 지연 로딩 적용

815.10 kB → 204.38 kB

- [gzip 기준]
- 동일 기준 약 74.9% 감소

123 files / 657 tests

- [검증 기록]
- 관련 작업에서 남긴 최대 확인 기록

범위

- 물류 조회, 예약 접수, 현황, 주소록 화면
- 반응형 테이블, 중첩 모달, 날짜/드롭다운 UI
- 엑셀 업로드/다운로드, 미리보기, 전송 데이터 변환
- PC 출력과 모바일 WebView 출력 경로

말은 일

- 초기 진입에 바로 필요하지 않은 화면과 엑셀 처리 코드를 라우트 단위로 분리했습니다.
- 모의 데이터와 실제 API 어댑터 경계를 나눠 화면 상태와 연동 상태를 따로 확인할 수 있게 했습니다.
- PC 브라우저, 일반 모바일 브라우저, 모바일 앱 WebView의 출력 분기를 명확히 나눴습니다.

결과

- 라우트 분리와 엑셀 처리 라이브러리 지연 로딩으로 초기 로드 번들을 2,405.50 kB에서 616.59 kB로 줄였습니다.
- 가로 넘침, 모달 잘림, 드롭다운 위치 문제를 화면 폭별로 확인했습니다.
- 조회, 예약, 출력이 이어지는 물류 업무 흐름을 한 화면 체계 안에서 설명할 수 있게 됐습니다.

검증

- 390px~1366px 주요 화면 반응형 확인
- 모달, 테이블, 드롭다운 표시 상태 확인
- 관련 작업 기준 123 files / 657 tests 기록



모바일 라벨 출력 앱

2026.04 ~ 2026.06

물류 관리-운영 웹을 모바일 **WebView**로 열고, **프린터 SDK**를 네이티브로 연결했습니다.

물류 관리-운영 웹을 모바일 앱 **WebView**로 제공하면서, 모바일에서만 필요한 **Bluetooth** 프린터 SDK 연동을 네이티브 모듈로 연결했습니다. 단독 프린터 앱이라기보다 운영 웹의 모바일 출력 경로를 담당하는 앱에 가깝습니다.

기술 스택

Expo · Expo Router · React Native · React Native WebView · Android native module · Zustand

프로젝트 종류

Mobile

프로젝트 기반

신규 구축

Bluetooth 권한

- [모바일 출력 준비]
- 최근 Android 런타임 권한 흐름 확인

개발/운영 분리

- [앱 배포 흐름]
- URL, 앱 식별자, 설치 파일 기준 정리

실기기 출력

- [프린터 SDK 검증]
- 에뮬레이터가 아닌 실기기 출력 결과 확인



범위

- 운영 웹 **WebView** 진입과 개발/운영 URL 분리
- JavaScript-to-native 출력 데이터 계약
- Android 네이티브 **Bluetooth** 출력 모듈
- 개발/운영 설치와 업데이트 확인 흐름



말은 일

- **WebView** 브리지 요청과 네이티브 출력 모듈 사이의 요청/응답 계약을 정리했습니다.
- **Bluetooth** 장비 탐색, 상태 확인, 출력 명령의 네이티브 흐름을 연결했습니다.
- Android 실기기에서 로컬 개발 서버와 API 요청이 잘못된 로컬 루프백으로 향하는 문제를 보정했습니다.



결과

- 웹 전송 데이터와 네이티브 출력 레이아웃을 분리해 이후 라벨 필드 추가 위험을 줄였습니다.
- 출력 기능을 웹 버튼이 아니라 실제 모바일 장비 연동으로 검증했습니다.
- 개발/운영 설치 기준을 맞춰 현장 설치 흐름을 설명할 수 있게 했습니다.



검증

- 실제 Android 기기에서 **Bluetooth** 출력 확인
- 최근 Android 권한 흐름 재현 및 수정 확인
- **WebView** bridge와 네이티브 출력 요청 흐름 확인



프로젝트 착수 문서화 도구

2026.04

착수 자료와 저장소 근거를 바탕으로 문서 초안의 생성 도구를 구축했습니다.

프로젝트 착수 시 전달받는 자료와 로컬 저장소 근거를 바탕으로 요구사항, 기능, 화면 단위의 문서 초안을 만드는 도구를 구축했습니다. AI API는 생성과 일부 재작성 단계에 쓰고, 그 전에 규칙 기반 스캔으로 근거를 먼저 모으는 구조로 잡았습니다.

기술 스택

Node.js · TypeScript · React · AI Agent workflow · Vite · pnpm · Vitest

프로젝트 종류

Tooling

프로젝트 기반

신규 구축

근거 스캔 → AI 초안

- [생성 흐름]
- 스캔 근거를 바탕으로 문서 초안 생성

xlsx 워크북

- [시트형 산출물]
- 요구사항/기능/화면 단위를 표 형태로 검토

AI 기반 셀 재작성

- [검토 후 수정]
- 선택 시트/셀 기준으로 일부 항목 재생성

범위

- 로컬 저장소 스캔과 evidence JSON 생성
- 미리보기, 상세 산출물, 이전 실행 결과 재열기
- 요구사항 원장, 기능 정의서, 화면 설계서 워크북 편집
- Markdown, JSON, ZIP, xlsx 내보내기

맡은 일

- 규칙 기반 스캐너 결과를 생성 단계의 입력으로 사용하도록 흐름을 나눴습니다.
- 긴 텍스트 문서뿐 아니라 표 기반 검토와 수정이 가능하도록 워크북 편집을 연결했습니다.
- 사람이 검토한 뒤 특정 셀만 다시 쓰는 흐름을 만들어 수정 단위를 줄였습니다.

결과

- 착수 자료 분석과 문서 초안 작성을 반복 가능한 실행 단위로 만들었습니다.
- 생성 결과를 그대로 쓰는 구조가 아니라 근거, 미리보기, 사람 검토, 내보내기가 이어지는 흐름으로 만들었습니다.
- 신규 프로젝트를 시작할 때 요구사항과 화면 단위를 빠르게 잡기 위한 내부 도구로 정리했습니다.

검증

- 자료 스캔, AI 초안 생성, 워크북 내보내기 흐름 확인
- 선택 셀 재작성과 이전 결과 재열기 확인
- Markdown/JSON/ZIP/xlsx 내보내기 확인



차트 편집 웹 도구

2026.03 ~ 2026.04

차트 미리보기, 옵션 편집, WebGL/GLSL 배경 렌더링을 포함한 편집 도구를 만들었습니다.

데이터 필드 매핑, 차트 미리보기, 옵션 패널, 드래그 앤 드롭, 툴팁 같은 편집 흐름을 구축했습니다.

차트 출력 화면이 아닌, 사용자가 차트 구성을 직접 조정하는 편집 도구에 가까운 작업이었습니다.

기술 스택

React · TypeScript · Vite · Chart.js · WebGL/GLSL (OpenGL 계열) · OGL (WebGL 라이브러리) · react-chartjs-2 · TanStack Query · Zustand · MSW · Vitest

프로젝트 종류

Web

프로젝트 기반

신규 구축

6개 탭 설정 패널

- [설정 구조]
- 차트 설정을 영역별로 분리

Chart.js + WebGL/GLSL

- [렌더링 기반]
- 차트 미리보기와 OGL(WebGL) 기반 편집 화면 배경

Vitest / MSW

- [검증 기반]
- 상호작용과 데이터 흐름 확인



범위

- 필드/값 매핑과 차트 미리보기
- 옵션 패널, 접힘 패널, 툴팁
- 드래그 앤 드롭 기반 편집 UX
- WebGL/GLSL 셰이더 기반 편집 화면 배경 렌더러



말은 일

- 차트 렌더링과 설정 패널 상태가 따로 놀지 않도록 옵션 모델을 정리했습니다.
- 편집 중 사용자가 현재 설정을 이해할 수 있게 미리보기와 옵션 라벨을 맞췄습니다.
- WebGL/GLSL fragment shader로 여러 배경 모드를 구성하고 pointer/time/resolution uniform을 사용해 편집 화면의 시각 요소를 만들었습니다.



결과

- 단순 차트 렌더링이 아니라 조작 가능한 편집 도구 경험으로 정리했습니다.
- 시각화, 상태 관리, 복잡한 설정 UI, WebGL shader 기반 배경 렌더링을 함께 다룬 케이스입니다.
- 차트 라이브러리 사용을 넘어 옵션 모델과 편집 UX를 연결한 경험을 드러낼 수 있습니다.



검증

- 미리보기 렌더링, 옵션 변경, 패널 접기 흐름 확인
- 래그 앤 드롭, 툴팁, 필드 매핑 동작 확인
- 모의 데이터 기반 흐름 확인

동해선장

captain-donghae

2024.10

프로젝트 개요

동해선 기차 이용객을 위한 실시간 정보, 대중교통 경로, 러닝/자전거 추천 코스, 주변 맛집 정보 등을 제공하는 종합 가이드 웹 서비스.

기술 스택

TypeScript · React · Next.js · Tailwind CSS · Google Maps ·
Storybook · AWS · Github Actions · Docker

관련 링크

https://github.com/Busan-Trail/busan_trail_front

프로젝트 담당

프론트엔드 개발

프로젝트 인원

3인

프로젝트 기여

1. 구글 맵스 플랫폼 기반 동해선 서비스 구현

- 다양한 지도 API 중 대중교통 경로 안내, 마커 커스터마이징 등을 비교 분석하여 구글 맵스 플랫폼을 선정하였습니다.
- Maps JavaScript API를 활용한 동적 지도, Places API 기반 장소 검색, Directions API 활용 대중교통 경로 안내, Geocoding API 기반 주소 변환 기능을 구현하였습니다.
- Static Maps API를 활용하여 특정 위치의 정적 지도 이미지를 제공하고, Geolocation API 연동으로 실시간 위치 추적 기능을 구현하였습니다.

2. 외부 API 통합 및 데이터 서비스 구축

- axios를 활용한 API 모듈화와 Next.js API Routes를 통해 구글 맵스 API, 날씨, 지하철역 정보(실시간 혼잡도 포함), 러닝/자전거 코스, 맛집 정보 등 다양한 외부 API를 통합 구현하였습니다.
- 각 API 요청에 대한 처리(위도/경도, 페이지네이션, 카테고리 필터링 등)와 에러 상황 대응을 통해 안정적인 데이터 서비스를 제공하였습니다.



동해선장

captain-donghae

2024.10

프로젝트 기여

3. 컴포넌트 아키텍처 설계 및 UI 개발

- 합성(Composition) 패턴을 적용하여 기본 기능을 가진 공통 컴포넌트와 이를 확장한 구체적인 컴포넌트를 설계하였습니다.
- Storybook을 활용한 컴포넌트 주도 개발로 각 컴포넌트의 기능과 디자인을 체계적으로 개발하였습니다.
- 계층형 폴더 구조와 명확한 네이밍을 통해 베이스 컴포넌트와 확장된 컴포넌트 간의 관계를 직관적으로 파악할 수 있도록 구성하였습니다.
- 기능과 스타일링의 관심사를 분리하여 유연하고 재사용 가능한 컴포넌트 구조를 확립하였습니다.

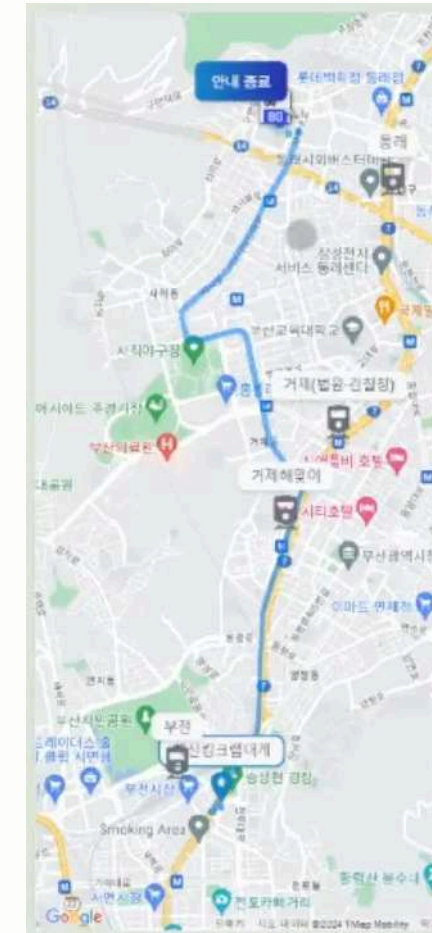
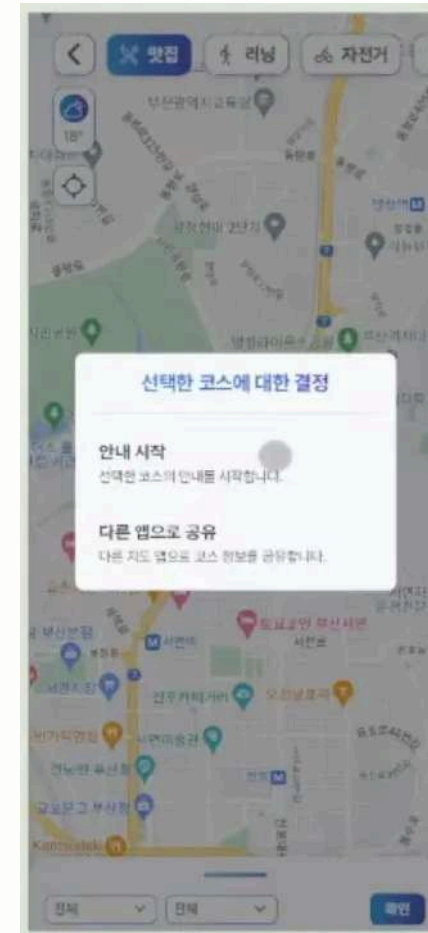
4. 인터랙티브 모달 컴포넌트 구현

- react-spring과 use-gesture를 활용하여 바텀 시트 모달의 부드러운 애니메이션과 제스처 인터랙션을 직접 구현하였습니다.
- 드래그 거리와 속도에 따른 단계별 위치 조정(10%, 30%, 60%, 85%, 92%)을 구현하여 직관적인 사용자 경험을 제공하였습니다.
- 스크롤과 드래그 상태를 동시에 관리하여 자연스러운 인터랙션이 가능하도록 구현하였습니다.

트러블슈팅

1. Next.js API 라우팅 경로 설정 오류 해결

- Next.js App Router에서 외부 Weather API 연동 시 API 라우팅 경로 불일치로 404 에러가 발생했습니다.
- route.ts 파일의 위치와 이름 그리고 폴더 구조를 Next.js 13 App Router 컨벤션에 맞게 수정하고, 요청 파라미터 처리 로직을 개선하여 문제를 해결했습니다.
- 에러 처리와 로깅을 강화하여 향후 비슷한 문제가 발생했을 때 빠른 디버깅이 가능하도록 개선했습니다.



동해선장

captain-donghae

2024.10

트러블슈팅

2. 지도 서비스 API 선정 및 구현 과정의 문제 해결

- 국내외 지도 API들의 웹 버전 기능을 비교 분석하여(카카오맵, 네이버맵, T맵, 구글맵) 프로젝트 요구사항에 가장 적합한 API를 검토하였습니다.
- 각 지도 API의 웹 버전 지원 기능(대중교통 API, 길찾기 API, 커스텀 스타일링)을 종합적으로 평가하여 구글 맵스 플랫폼을 최종적으로 선택하였습니다.
- Maps JavaScript API, Directions API, Places API, Static Maps API 등을 통합적으로 활용하여 완성도 높은 지도 서비스를 구현하였습니다.

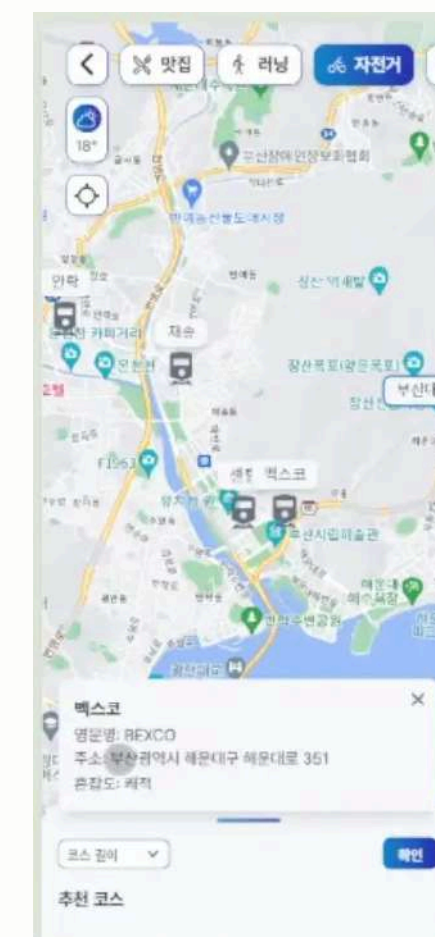
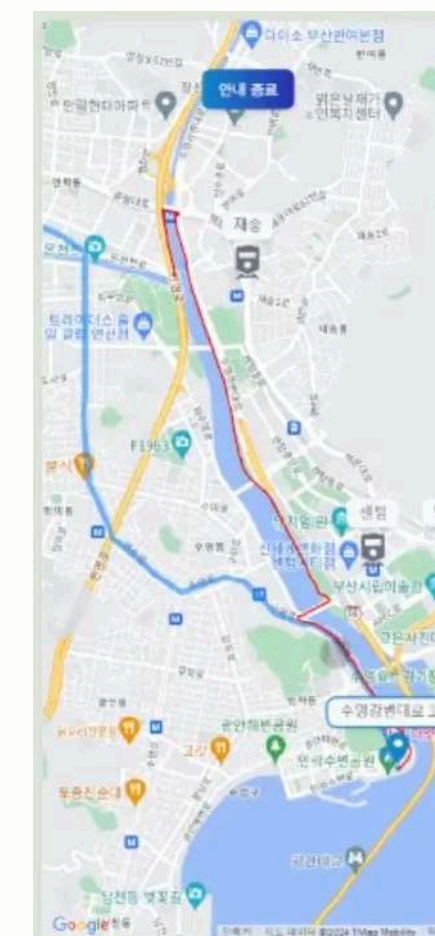
특별 사항

1. DIVE 2024 글로벌 데이터 해커톤 수상

- 부산광역시 주최, 부산테크노파크 주관의 DIVE 2024 글로벌 데이터 해커톤에서 부산테크노파크원장상(발제사 3위)을 수상하였습니다.
- 코레일(한국철도공사)의 데이터를 활용하여 동해선 이용객을 위한 가이드 서비스 "동해선장"을 개발하였습니다.
- 3인 팀의 유일한 프론트엔드 개발자로 참여하여, 72시간이라는 제한된 시간 내에 Swagger 문서 기반의 백엔드 API 연동 및 프론트엔드 개발을 수행하였습니다.
- 구글 맵스 플랫폼 연동, 인터랙티브 UI 구현, 실시간 데이터 통합 등 프론트엔드 개발 전반을 담당하였습니다.

2. 효율적인 팀 협업 체계 구축

- 프론트엔드 1명, 백엔드 2명으로 구성된 팀에서 원활한 협업을 위한 커뮤니케이션 체계를 구축하였습니다.
- Discord를 통한 주 3회 정기 회의와 Notion을 활용한 기획 문서화로 체계적인 프로젝트 관리를 진행하였습니다.
- GitHub Actions와 AWS, Docker를 활용한 CI/CD 파이프라인 구축으로 자동화된 배포 환경을 구성하였습니다.





잇집

Itzip

2024.07 ~ Present

프로젝트 개요

개발자 취준생을 위한 종합 취업 준비 플랫폼으로, 블로그, 테스트, 구인 정보 등을 제공하는 웹 서비스.

기술 스택

TypeScript · React · Next.js · Tailwind CSS · Jotai · Jest · Prisma ·
Storybook · Postman · Sentry · AWS · Jenkins · Docker

관련 링크

https://github.com/ITZipProject/itzip_front

프로젝트 담당

프론트엔드 개발, DevOps

프로젝트 인원

15인

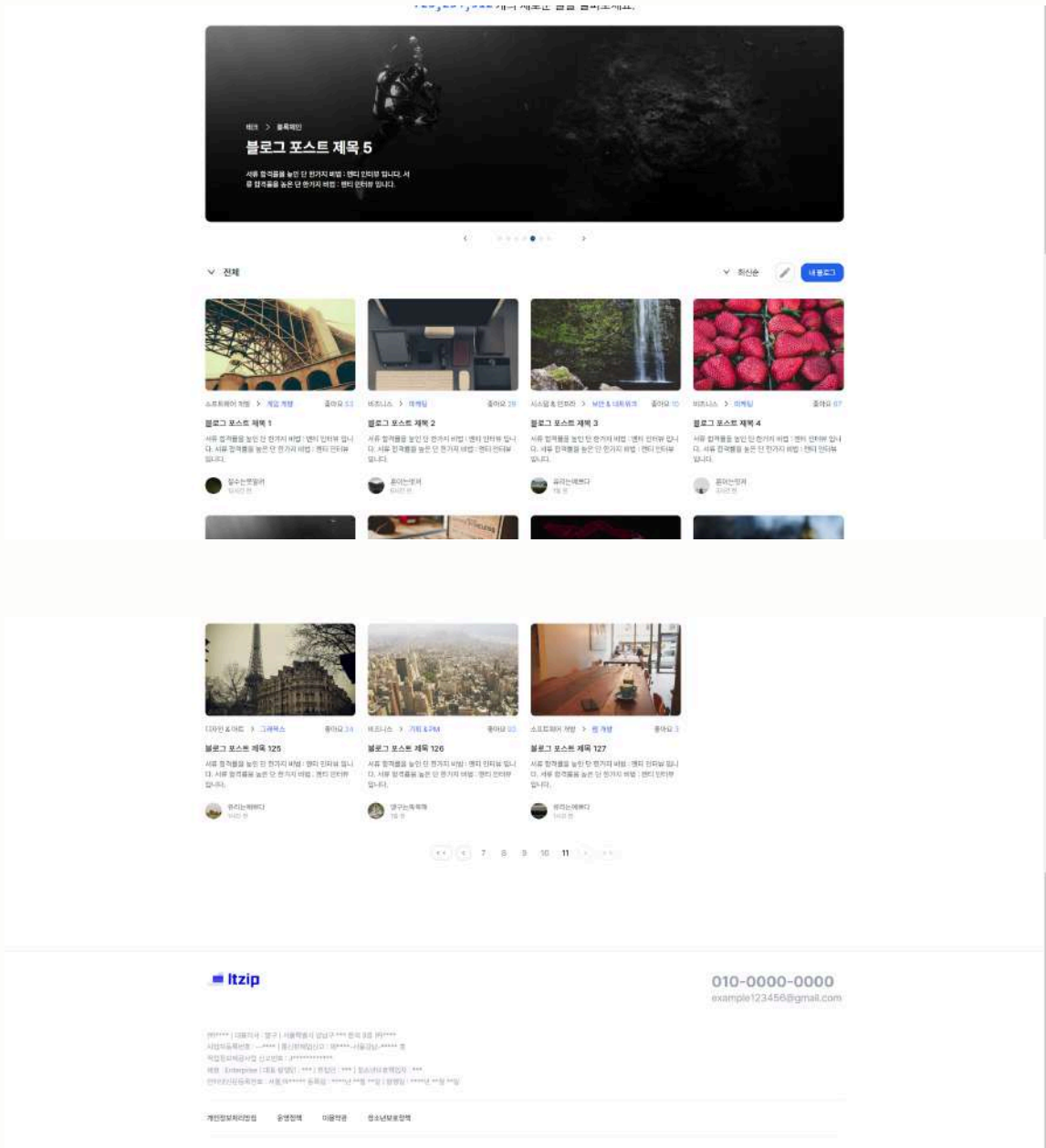
프로젝트 기여

1. 블로그 시스템 구현

- react-spring을 활용한 애니메이션 효과로 전체 글 개수를 슬롯머신 스타일로 표시하여 사용자 경험을 향상시켰습니다.
- 커스텀 캐러셀 컴포넌트를 직접 구현하여 메인 페이지의 시각적 매력을 높였습니다.
- 다양한 필터링 및 정렬 옵션을 제공하여 사용자가 원하는 콘텐츠를 쉽게 찾을 수 있도록 하였습니다.
- 페이지네이션 구현 시 사용자 편의를 고려하여 페이지 번호 클릭 시 화면 상단으로 자동 스크롤 되도록 하였습니다.

2. Markdown 에디터 개발

- 실시간 미리보기 기능을 갖춘 Markdown 에디터를 구현하였습니다.
- 커스텀 Markdown 문법 지원으로 풍부하고 편리한 콘텐츠 작성 환경을 제공하였습니다.





잇집

Itzip

2024.07 ~ Present

프로젝트 기여

3. 코드 품질 향상

- Jest를 이용한 단위 테스트와 Storybook을 활용한 컴포넌트 문서화로 코드 품질을 향상시켰습니다.
- Sentry를 통한 실시간 오류 모니터링 시스템을 구축하여 신속한 버그 대응이 가능하도록 했습니다.

4. DevOps 및 인프라 구축

- AWS를 활용한 확장 가능한 클라우드 인프라를 설계하고 구축하였습니다.
- Jenkins를 이용한 CI/CD 파이프라인을 구성하여 지속적 통합 및 배포 프로세스를 자동화하였습니다.
- Docker를 사용하여 서비스 컨테이너화를 진행하고, 개발 및 운영 환경을 구성하였습니다.

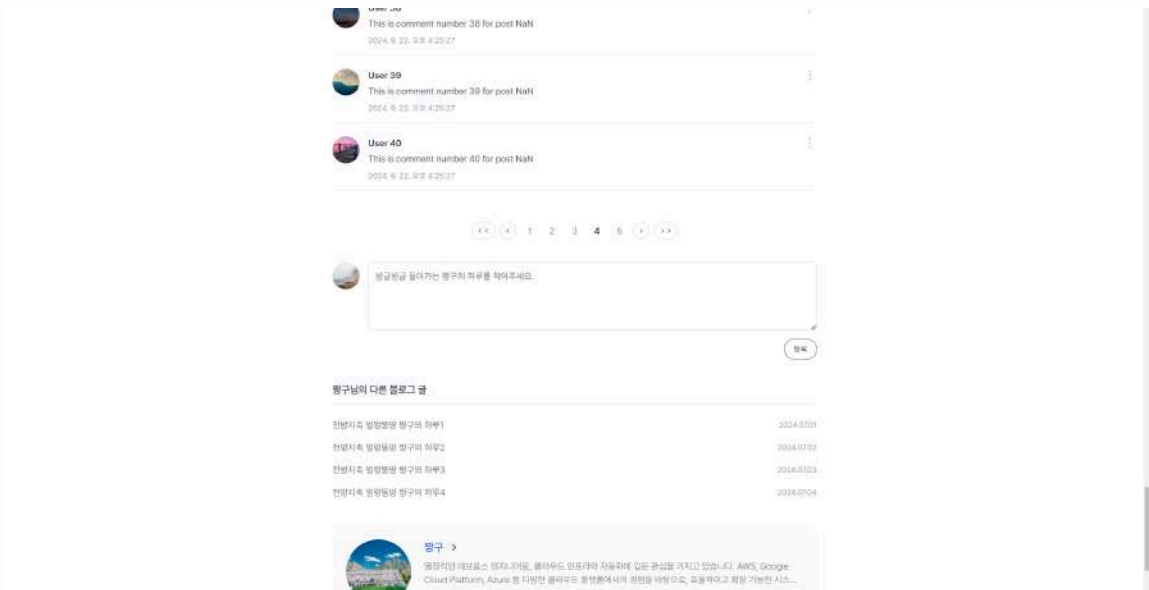
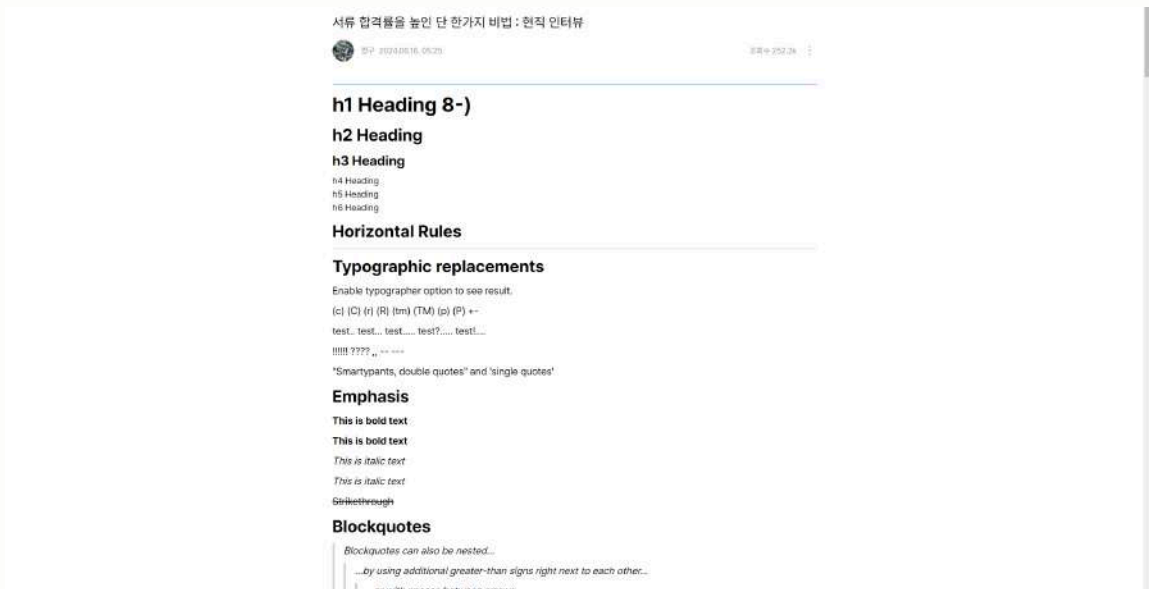
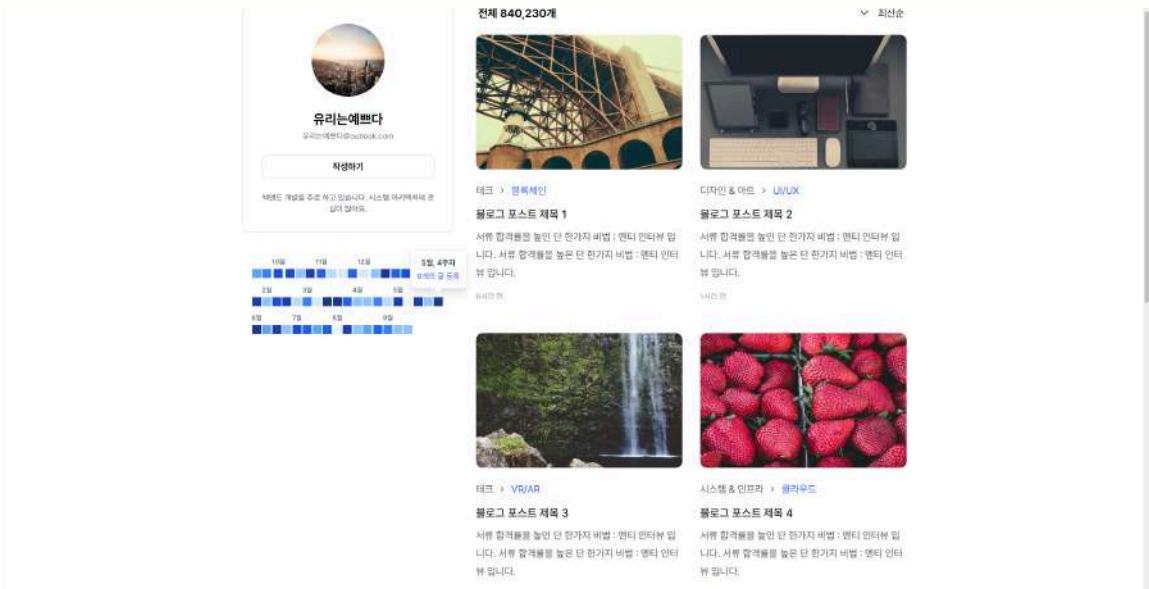
성능 개선

1. 이미지 최적화

- Next.js의 Image 컴포넌트를 활용하여 이미지 로딩 성능을 최적화하였습니다.

2. 동적 임포트를 통한 코드 스플리팅

- Next.js의 dynamic import를 활용하여 게시글 내부 컴포넌트들을 동적으로 로드하도록 구현했습니다.
- 이를 통해 초기 페이지 로드 시간을 단축하고, 필요한 시점에 관련 컴포넌트를 로드하여 전체적인 성능을 개선했습니다.



잇집

ltzip

2024.07 ~ Present

특별 사항

1. 확장 가능한 타이포그래피 시스템

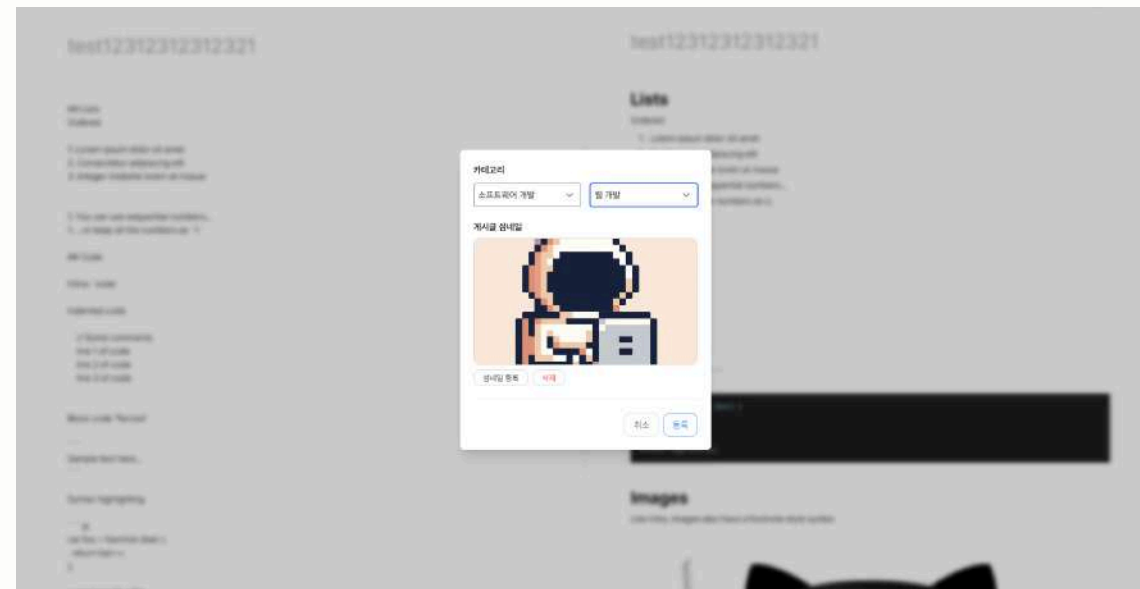
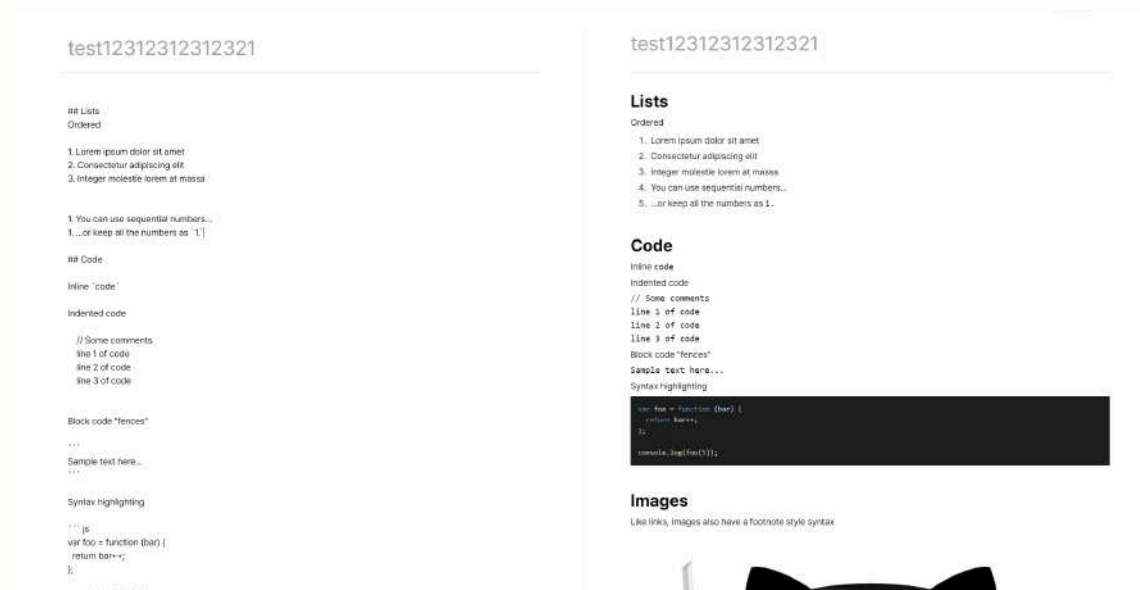
- Tailwind CSS를 확장하여 프로젝트 전반에 걸쳐 사용할 수 있는 일관된 타이포그래피 시스템을 구축했습니다.
- 반응형 디자인을 고려한 폰트 스케일링 로직을 개발하여, 화면 크기에 따라 자동으로 폰트 크기가 조절되도록 하였습니다.
- 이를 통해 디자인 일관성을 유지하면서도 다양한 디바이스에 최적화된 사용자 경험을 제공할 수 있게 되었습니다.

2. GitHub 스타일 기여도 그래프

- 사용자의 글쓰기 활동을 시각화하기 위해 GitHub의 기여도 그래프와 유사한 커스텀 컴포넌트를 개발하였습니다.
- 더 상세한 정보를 볼 수 있는, 마우스 호버 시 나타나는 툴팁 기능을 추가하였습니다.
- 이 기능을 통해 해당 사용자의 활동 패턴을 직관적으로 표현하고, 지속적인 참여를 유도하는 효과를 얻도록 하였습니다.

3. 효율적인 팀 협업 시스템 구축

- 15명 규모의 팀(프론트엔드 5명, 백엔드 5명, 디자이너 5명)에서 프론트엔드 팀장으로서 활동하였습니다.
- Notion, Discord, Slack을 활용한 체계적인 문서화 및 실시간 커뮤니케이션 시스템을 구축하여 원활한 정보 공유와 신속한 의사결정을 가능케 했습니다.
- 주간 회의를 통해 프로젝트 진행 상황을 공유하고 코드 품질을 지속적으로 개선하였습니다.
- Figma와 Swagger를 활용하여 디자인-개발 및 프론트엔드-백엔드 간 효율적인 협업 프로세스를 확립하였습니다.



쉐어비

ShareBBy

2024.04 ~ 2024.06

프로젝트 개요

다양한 취미 활동을 공유하고 참여할 수 있는 플랫폼을 제공하는 크로스플랫폼 앱.

기술 스택

JavaScript · React Native · Firebase · Faster Image

관련 링크

<https://github.com/jukrap/rn-ShareBBy>

<https://youtu.be/Zu1Git1zAAA>

프로젝트 담당

프론트엔드 및 백엔드 개발

프로젝트 인원

5인

프로젝트 기여

1. 안드로이드 작업

- IOS 기준으로 개발된 앱을 안드로이드에 맞도록 수정하고 개선하였습니다.

2. 데이터베이스 관련

- ERD를 제작하여 데이터 관계를 규정하고, Firebase에서의 구조를 설계하였습니다.

3. 게시판 전체 구현

- 사용자들이 편리하게 게시글과 댓글을 작성, 읽기, 수정, 삭제할 수 있도록 구현하였습니다.
- Firebase의 실시간 데이터베이스를 활용하여 실시간 댓글, 좋아요, 다중 이미지 등의 기능도 구현하였습니다.

4. 모달 및 토스트 제작

- 각종 모달 및 토스트 메시지를 제작하였습니다.



쉐어비

ShareBBy

2024.04 ~ 2024.06

트러블슈팅

1. 이미지 캐싱 이슈 해결

- 초기에 리액트 네이티브의 기본 이미지 컴포넌트 사용으로 인한 과도한 트래픽 및 느린 로딩 속도 문제를 확인하였습니다.
- fast-image 라이브러리 적용을 시도했으나, 업데이트 중단 이슈로 인해 faster-image 라이브러리를 대안으로 적용하였습니다.
- 이를 통해 이미지 캐싱 문제를 해결하고 앱의 성능을 크게 개선하였습니다.

성능 개선

1. 이미지 최적화

- 모바일 환경에 최적화된 UI를 고려하여 이미지 관련 작업을 수행하였습니다.
- 이미지 크기 조정 및 캐싱 등의 최적화를 통해 성능을 개선하였습니다.
- 이러한 최적화로 서버 트래픽이 기존 대비 30~50% 가량 감소하는 효과를 얻었습니다.

2. 게시판 및 댓글 성능 개선

- Pull to Refresh와 Infinite Scroll 구현으로 대량의 데이터를 효율적으로 로드할 수 있게 되었습니다.

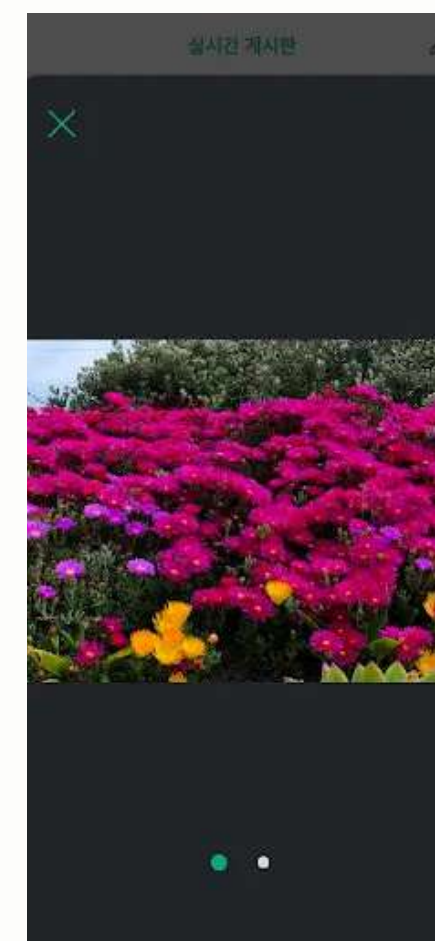
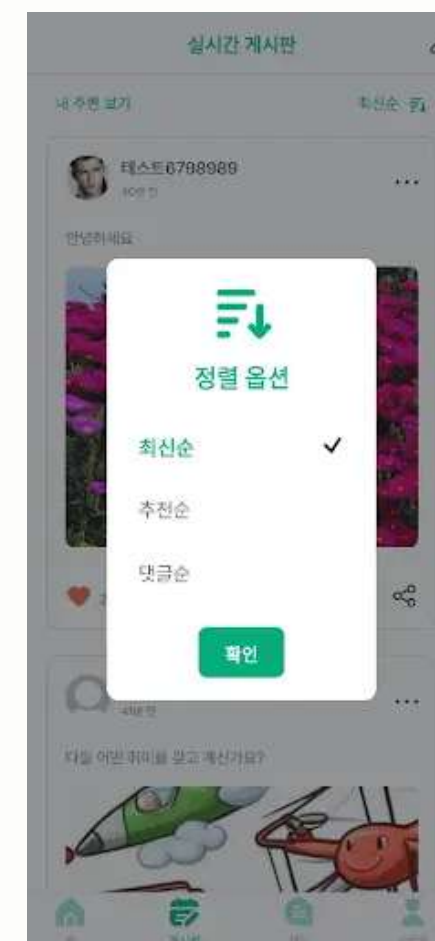
특별 사항

1. 위치 기반 게시글 필터링

- 사용자 위치를 기반으로 가까운 위치의 게시글만 표시하는 기능을 구현했습니다.
- 위치 데이터를 효율적으로 처리하여 사용자 경험을 향상시켰습니다.

2. 협업 도구 활용

- 스크럼 방법론을 통해 주기적인 소통과 개발 작업의 조율, 일정 관리를 진행하였습니다.
- Notion, Slack, Github, Figma 등의 협업 툴을 적극적으로 활용하여 팀 작업의 효율성을 높였습니다.



자세선생

Posture Teacher

2022.07 ~ 2023.06

프로젝트 개요

사람의 앉은 자세 혹은 플랭크 자세를 감지한 다음, 신체 각 지점의 각도와 길이에 따라 올바른 자세 여부를 판별하고 데이터를 제공하는 앱.

기술 스택

Java · Android · Jetpack · Mediapipe · SQLite

관련 링크

<https://github.com/jukrap/Posture-Teacher>

프로젝트 담당

팀장, 안드로이드 개발

프로젝트 인원

2인

프로젝트 기여

1. 타이머 기능 구현

- 자세 유지 시간과 어긋난 시간을 기록하기 위한 타이머 기능을 구현했습니다.
- 각 자세에 맞는 개별적인 타이머 시스템을 설계하고 구현했습니다.

2. Mediapipe 통합 및 최적화

- Mediapipe AAR를 리눅스 환경에서 빌드하고 프로젝트에 통합했습니다.
- Mediapipe를 사용하여 몸과 얼굴의 동작을 측정하는 솔루션을 개발했습니다.
- OpenCV 대비 5~10배 높은 FPS를 확보하여 성능을 크게 개선했습니다.

3. 신체 측정 기능 구현

- 앉은 자세와 플랭크 자세의 올바름을 측정하는 기능을 구현했습니다.
- 각 자세에 대한 측정 페이지를 설계하고 개발했습니다.

4. 데이터베이스 구축

- SQLite 기반의 Room을 사용하여 데이터베이스 구조를 설계하고 구축했습니다.
- Dao, Entity, Database를 설계하여 CRUD 작업을 최적화하였습니다.





자세선생

Posture Teacher

2022.07 ~ 2023.06

트러블슈팅

1. 멀티 스레드 최적화

- Mediapipe의 부하를 감소시키기 위해 Runnable 인터페이스와 스레드 클래스를 활용한 멀티 스레드 최적화를 수행했습니다.
- 이를 통해 구형 휴대폰에서의 앱 사용성을 개선하고 UI 반응성 문제를 해결했습니다.

2. 빌드 환경 문제 해결

- 구형 CPU&GPU 관련 문제로 인해 Docker와 MSYS2 사용이 중단되었습니다.
- 최종적으로 Ubuntu 환경에서 Mediapipe 빌드 작업을 성공적으로 수행했습니다.

성능 개선

1. FPS 개선

- Mediapipe 사용으로 OpenCV 대비 5~10배 높은 FPS를 달성했습니다.
- 이를 통해 실시간 자세 측정의 정확도와 반응성을 크게 향상시켰습니다.

프로젝트 기여

1. 체격 차이 고려 알고리즘

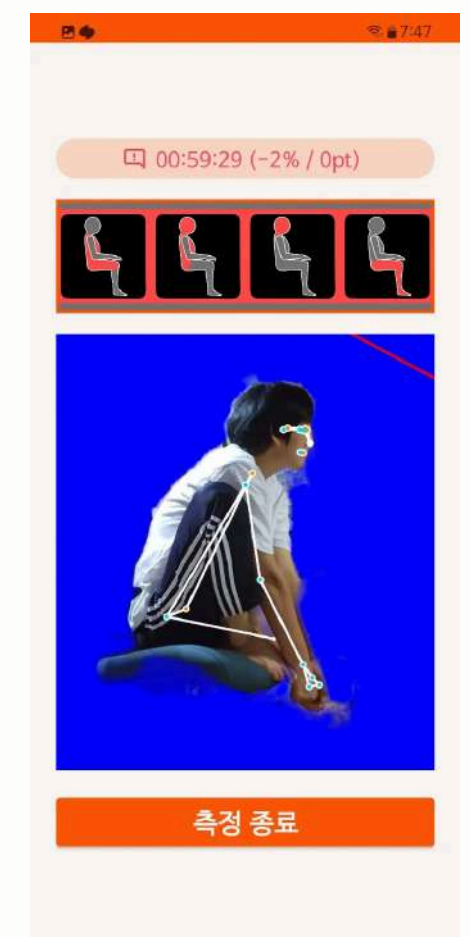
- 사용자 간 체격 차이를 고려한 자세 측정 알고리즘을 개발했습니다.
- 이를 통해 다양한 체형의 사용자에게 자세 피드백을 제공할 수 있었습니다.

2. 프로젝트 성과

- 경남소프트웨어경진대회에서 예선 통과 후 본선까지 진출하였습니다.
- 이후 구글 플레이스토어에 배포를 진행했습니다.

3. 애자일 스크럼 방식 프로젝트 진행

- 주 1-2회 정기적인 미팅을 통해 개발 진행 상황을 공유하고 다음 목표를 설정했습니다.
- 이러한 프로젝트 진행 방식 덕분에, 빠른 피드백과 일정한 팀 분위기를 유지할 수 있었습니다.



ESD 핫딜

ESD HotDeal

2021.07 ~ 2021.11

프로젝트 개요

여러 ESD에서 제공하는 할인, 무료 소프트웨어 목록을 정리해서 알려주는 웹서비스.

기술 스택

JavaScript · React · Firebase · Node.js · Express · Puppeteer

관련 링크

<https://github.com/jukrap/RollCakeProject>

프로젝트 담당

팀장, 프론트엔드 및 백엔드 개발

프로젝트 인원

2인

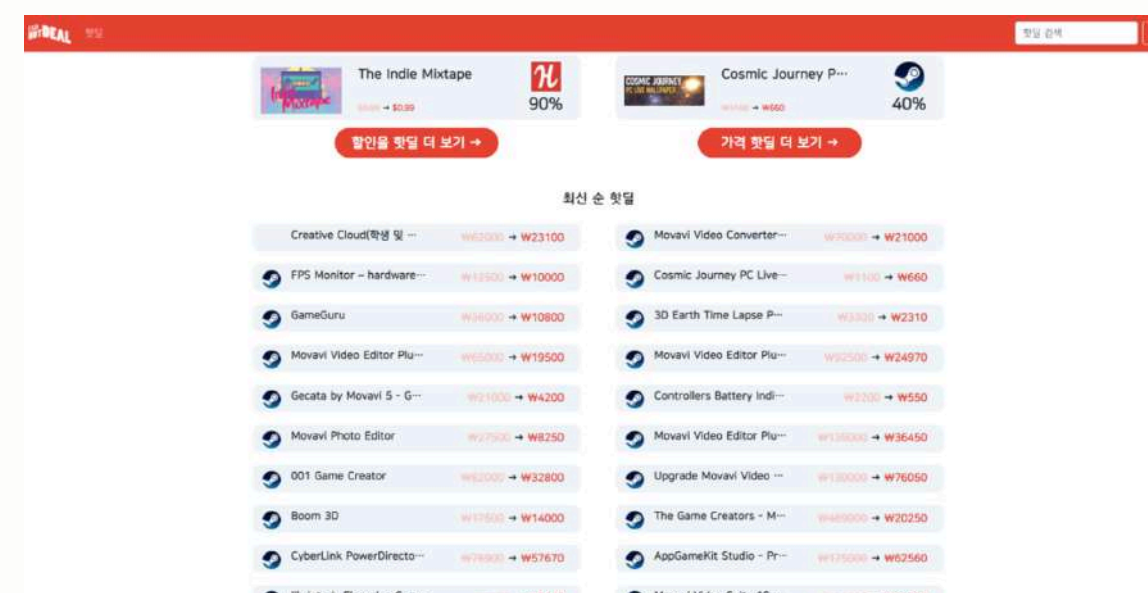
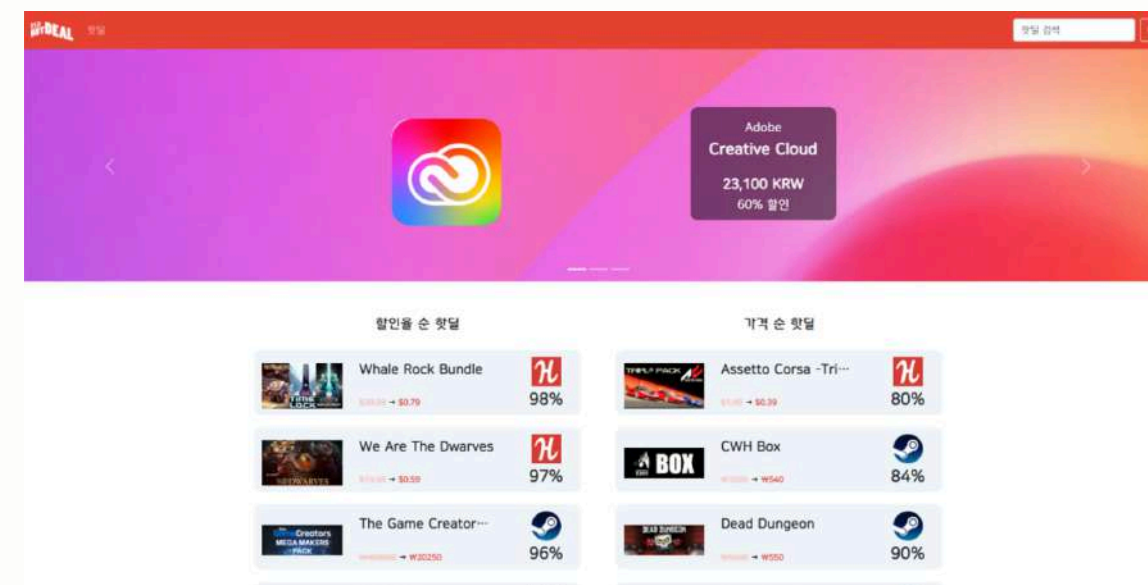
프로젝트 기여

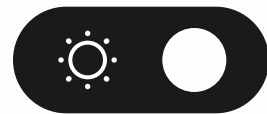
1. 프론트엔드 개발

- React를 기반으로 한 동적 웹 애플리케이션을 구현하였습니다.
- react-router-dom을 사용하여 라우팅 시스템을 구축하였습니다.
- react-bootstrap을 활용하여 반응형 디자인을 구현하였습니다.
- 메인 페이지, 핫딜 페이지, 검색 페이지 등 주요 페이지들의 레이아웃과 기능을 구현하였습니다.

2. 백엔드 개발

- Express를 사용하여 서버를 구축하고 백엔드를 설계 및 구현하였습니다.
- Puppeteer 라이브러리를 활용하여 효율적인 웹 크롤링 시스템을 개발하였습니다.
- 수만 건의 핫딜 데이터를 3~5분 내에 크롤링 및 가공하는 시스템을 구현하였습니다.
- Firebase를 활용하여 문서 기반의 NoSQL 데이터베이스를 설계하고 구축하였습니다.





ESD 핫딜

ESD HotDeal

2021.07 ~ 2021.11

프로젝트 기여

3. 재사용 가능한 컴포넌트 개발

- styled-components 기반의, 여러 페이지에서 공통적으로 사용되는 상품 정보 컴포넌트를 제작하였습니다.
- ESD 사이트로 연결되는 하이퍼링크 기능을 포함한 컴포넌트를 구현하였습니다.
- 이를 통해 개발 효율성을 높이고 일관된 UI/UX를 제공할 수 있었습니다.

트러블슈팅

1. 정적 웹 호스팅 문제 해결

- 초기에 Github Pages를 통한 정적 웹 호스팅을 시도하였으나, React SPA의 동적 특성으로 인해 문제가 발생하였습니다.
- 이 과정에서 정적 웹과 동적 웹의 차이점을 명확히 이해하게 되었습니다.
- 최종적으로 동적 웹 호스팅 서비스를 활용하여 문제를 해결하였습니다.

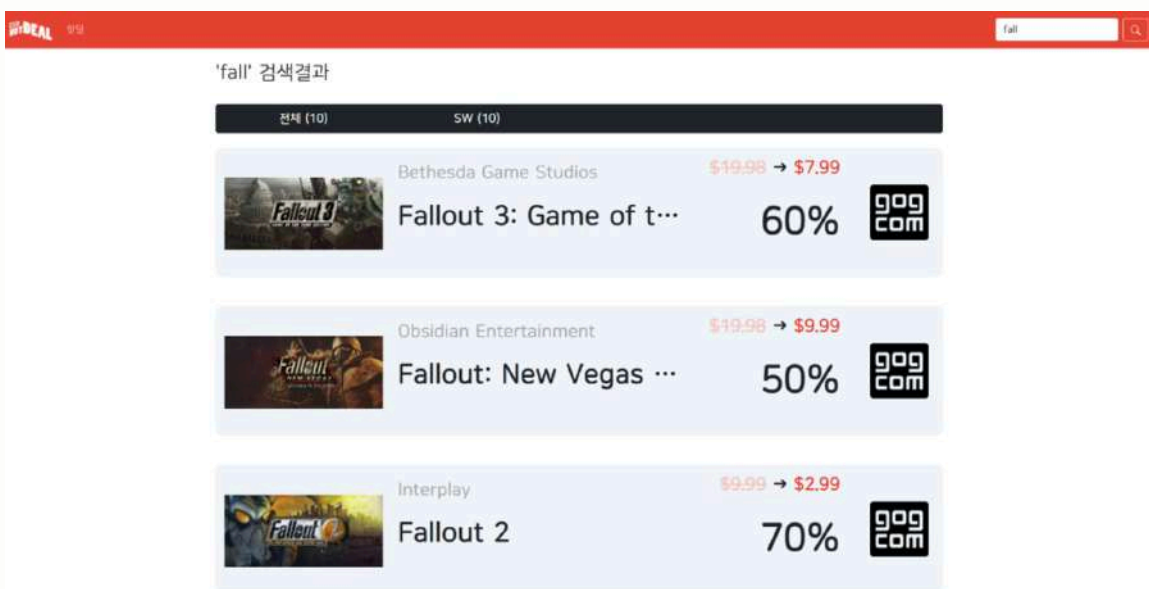
특별 사항

1. 프로젝트 성과

- 경남소프트웨어경진대회에서 최우수상을 수상하였습니다.

2. 효율적인 협업 시스템 구축

- 주간 스프린트를 설정하여 단기 목표를 명확히 하고, 빠른 피드백 사이클을 통해 변화에 신속하게 대응하였습니다.
- 스탠드업(Stand-Up) 미팅을 통해 팀원 간 진행 상황을 공유하고 장애 요소를 조기에 식별하여 해결하였습니다.
- 2인 팀으로 축소된 상황에서도 체계적인 일정 관리와 효율적인 작업 분배를 통해 프로젝트를 성공적으로 완수하였습니다.



끝.

읽어주셔서 감사합니다!